DP-GRM Auditor API Documentation

Last updated: November 2, 2026.

1 Preliminaries and Dependencies

The `DPGRMAuditor` Python module provides an interface for auditing **D**ifferentially **P**rivate **G**raph **R**elease **M**echanisms on user-supplied datasets and mechanisms, using the methodology presented in D'Silva et al. in [2], based on the statistical procedure introduced by Jagielski et al. in [7]. The source code is available at the GitHub repository[1] under the MIT license. This document contains the API documentation for this module. The dependencies, excluding those from the Python standard library are:

- numpy [5]
- scipy [8]
- matplotlib [6]
- tqdm [1]
- typeguard [3]
- The example metrics given require NetworkX [4].

Throughout, the following abbreviations are used:

- A: the DP-GRM being audited
- G, G': a neighbouring pair of input graphs to A
- $\tilde{G}$: the output graph of A
- (ε, δ): the DP parameters that were set when executing A
- t: the number of trials for selection *plus* evaluation

The main module exists as `auditor.py`. Example usage is provided in `example_usage.py` and `example_metrics.py`.

If you notice any bugs or inconsistencies with the module or this document, please create an issue on the GitHub repository.

[1] https://github.com/ndsi6382/DP-GRM_Auditor

2 API

The module is used by creating a `DPGRMAuditor` object.

`DPGRMAuditor(...)`, the object constructor, expects all of the following arguments (unless otherwise specified) in the given order:

- `pairs_info`: *may be supplied as any of the following:*
  (a) *list[tuple(string, string)]*; or
  (b) *list[tuple(string, 0/1 int)]*; or
  (c) *tuple(tuple(string, string), tuple(string, string), int)*

  The supplied option specifies how the audit should be run:

  (a) A list of tuples of length t, where the i-th tuple contains a pair of file paths, the first referring $A(G)$, and the second to $A(G')$ for the i-th trial.
  (b) A list of tuples of length t, where the i-th tuple contains: (i) a file path to $\tilde{G}$; and (ii) an integer label (0 or 1), specifying whether $\tilde{G} \sim A(G)$ or $\tilde{G} \sim A(G')$ for the i-th trial. While this halves the computation required (compared to (a)), it is important that, across trials, the choice of whether to execute A on G or G' remains independent and uniformly random. It is the user's responsibility to ensure this for the audit to remain valid, as the module cannot verify this.
  (c) A tuple containing three elements: (i) a tuple containing the shell command for executing A from the CLI on G, and the path to its output graph; (ii) another tuple containing the shell command for executing A from the CLI on G', and the path to its output graph; and (iii) t, the total number of trials. Note this live-executes the mechanism A a total of t times as part of the audit procedure, thus may take longer than the other two options, which assume that this execution step has already been performed.

- `epsilon_claimed`: *float*

  The ε used by the audited mechanism. Must be ≥ 0.

- `delta_claimed`: *float*

  The δ used by the audited mechanism. Must be in $[0, 1]$. If the mechanism claims to provide a pure-DP guarantee, then this should be set to 0.

- `confidence`: *float*

  Specifies the confidence level to be used for the statistical procedure. Must lie within $[0, 1]$.

- `metric_fn`: *function: string $\rightarrow$ float*

  The witness statistic to be used for the current run of the audit. The function is expected to accept a string for opening the path specified in `pairs_info`. It opens the object, performs the measurement, and returns a numeric (integer or float) value. Partial functions may also be used for parameterised algorithms, e.g., top k-th degree; for example usage, refer to the next section.

- `selection_evaluation_split`: *float*

  How to split the total number of trials into selection and evaluation trials. Must be within $[0, 1]$. Expresses the proportion of trials to be used for the *selection* phase.

- Optional: `seed`: *int*

  Whether to set a seed value for randomised operations. By default, this is not set.

`DPGRMAuditor.run(verbose)` runs the audit.

Runs the audit using the parameters set when initialising the `DPGRMAuditor` object.

Returns a *dict* of results.

- Optional: `verbose`: *bool*

  If set to `True`, the audit will print progress and results to the terminal. By default, this is `True`.

`DPGRMAuditor.plot(metric_name, title)`

Plots and visualises the results obtained in the most recent audit. This can only be used if the `.run()` method has already been executed.

Returns *(fig, axes)* objects as per the `matplotlib` library. Additionally, a `.pdf` figure is saved in the current working directory.

- Optional: `metric_name`: *string*

  The name of witness metric used for the audit. This is used for setting subplot titles. If this is not set, then the metric is not specified in the subplot.

- Optional: `title`: *string*

  The title of the plot. This is used for setting the overall title, and for setting the file-name of the saved `.PDF` figure. If this is not set, the title will be empty, and the `.PDF` figure will be saved as `plot.pdf`

3 Example Usage

The module can be used by importing the `DPGRMAuditor` class from `auditor.py`, initialising it with the desired parameters, then running the audit and optionally plotting the results:

```
from auditor import DPGRMAuditor

a = DPGRMAuditor(pair_info, 0.1, 1e-5, 0.95, f)
a.run()
a.plot()
```

Where `pair_info` may have been defined like any of:

```
# option (a)
pair_info = [(path_0_1, path_1_1), (path_0_2, path_1_2), ...,
             (path_0_t, path_1_t)]
    # where path_i_j points to the output for the mechanism's execution that was:
        # on the i-th canary (i=0 or i=1); and
        # for the j-th trial

# option (b)
pair_info = [(path_1, label_1), (path_2, label_2),..., (path_t, label_t)]
    # paths are strings, labels are 0/1 integers

# option (c)
pair_info = ((cmd_1, path_1), (cmd_2, path_2), num_trials)
    # cmds execute the mechanism, e.g. "mechanism.py --dataset 0 --out ./0.txt"
    # paths express where the output for that command is, e.g. "./0.txt"
```

And where `f` may have been defined like:

```
# Example: edge count
def f(path):
    with open(path, 'rb') as file:
        G = pickle.load(file)
    return G.number_of_edges()
```

Note, input canary pairs must be generated prior to running the audit. It is the user's responsibility to construct valid pairs that correctly align with the neighbouring relation used by the mechanism being audited.

Furthermore, if using options (a) or (b) to construct `pair_info`, then the mechanism must first be run for the specified number of trials on the inputs, with each output recorded for the module to access during the audit.

Using Partial Functions as Witnesses

Suppose you wish to use a parameterised statistic as the witness, such as the top k-th degree. Then, `f` can be defined and passed in as `functools.partial` function as follows (using NetworkX as an example):

```
import pickle
from functools import partial
```

```
def top_kth_degree(path, k):
    with open(path, 'rb') as file:
        G = pickle.load(file)
    deg = [d for v, d in G.degree()]
    deg.sort(reverse=True)
    return deg[k]

# set the value for k and initialise f
k = 2
f = partial(top_kth_degree, k)

# then run the audit
a = DPGRMAuditor(pairs, 0.1, 1e-5, 0.95, f)
```

References

[1] Casual Programmer's Incremental Developments. tqdm: A Fast, Extensible Progress Bar for Python. https://github.com/tqdm/tqdm.

[2] D'Silva, N., Nepal, S., and Kanhere, S. S. Practical auditing of differentially private graph release mechanisms: Methodology, toolkit, and case study, 2026.

[3] Grönholm, A. Typeguard. https://github.com/agronholm/typeguard, 2015.

[4] Hagberg, A. A., Schult, D. A., and Swart, P. J. Exploring network structure, dynamics, and function using networkx. *Python in Science Conference* (2008).

[5] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., and Oliphant, T. E. Array programming with NumPy. *Nature 585*, 7825 (Sept. 2020), 357–362.

[6] Hunter, J. D. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering 9*, 3 (2007), 90–95.

[7] Jagielski, M., Ullman, J., and Oprea, A. Auditing differentially private machine learning: How private is private sgd? In *Advances in Neural Information Processing Systems* (2020), H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33, Curran Associates, Inc., pp. 22205–22216.

[8] Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., Carey, C. J., Polat, İ., Feng, Y., Moore, E. W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E. A., Harris, C. R., Archibald, A. M., Ribeiro, A. H., Pedregosa, F., van Mulbregt, P., and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods 17* (2020), 261–272.